信息安全培训 · 60 分钟演讲稿

> **使用说明**
>
> - 讲稿按**调整后的页面顺序**编写（详见每页标题前的"原 slide N"）
> - 每页都标注了**建议用时**，60 分钟整体节奏：开场 2 分钟 + 第一部分 13 分钟 + 第二部分 17 分钟 + 第三部分 26 分钟 + Q&A 引入 2 分钟
> - 加粗部分是**关键术语和数字**，建议讲到时停顿 1 秒强调
> - 「⏸」符号表示**可以停下来与听众互动**或给思考时间的位置

开场（slide 1，约 2 分钟）

各位同事下午好，欢迎大家来参加这次信息安全培训。

我先简单交代一下这次培训的背景——为什么我们现在要花一个小时坐下来专门聊"信息安全"这件事。原因其实很直接：**2026 年 1 月，国标 GB 44495 已经强制实施**，这意味着我们做的任何一台车，如果信息安全做不到位，根本拿不到型式认证，连上市的资格都没有。这不是"做得好"的问题，而是"做不做得了"的问题。

那今天我想跟大家分享的，**不是堆砌一堆名词让大家记**，而是希望大家听完之后能建立三件事的认知：

第一，**信息安全到底在防什么**——它不是简单的"防黑客"，背后有一整套理论框架；
第二，**业界有哪些核心技术**在解决这些问题——加密、签名、PKI、安全启动这些到底在干嘛；
第三，**落到我们自己手上的角雷达 ECU**，从硬件、芯片、密钥、镜像、调试、生命周期，**端到端怎么做安全**。

整个流程大概 60 分钟，最后留 10 分钟左右给大家提问。中间有疑问随时打断我，不用等到最后。我们开始。

目录页（slide 2，约 15 秒）

整个分享分三大块：信息安全基础、信息安全核心技术、角雷达信息安全。前两块是给大家打底，**真正跟我们日常工作密切相关的是第三块**，所以前面我会讲得快一点，把时间留给后面。

第一部分 · 信息安全基础（约 13 分钟）

进入第一章（slide 3，约 5 秒）

进入第一章，信息安全基础。

概念定义（原 slide 4，约 1.5 分钟）

我们先把"汽车信息安全"这个词的定义讲清楚。

我喜欢用一个生活类比来理解它——**汽车现在已经不是一台机械车了，它就像你的家**。家里有门锁防陌生人进来（这是机密性）、有报警器防有人改东西（这是完整性）、有水电气保证你能正常生活（这是可用性）。汽车也一样，只不过它的"家"里面跑的是几百个 ECU、几亿行代码、上百路通信总线。

所以官方定义是这样：**汽车信息安全是指保护汽车及其相关系统免受未经授权的访问、使用、披露、破坏、修改或销毁的实践。**这句话拆开看就是六个动词——访问、使用、披露、破坏、修改、销毁，**这六个动作我们都要防。**它涵盖的范围也很广，从车辆硬件到软件，从通信网络到数据隐私，是一个全方位的概念。

大家先把这个"家"的类比记住，后面讲到 CIA 的时候我会反复回到这个比喻。

CIA + AAA 基本原则（原 slide 5，约 2 分钟）

刚才提到了 CIA，这页就是信息安全最核心的六个原则——**CIA 加上 AAA。**

CIA 是基础三性：

- **C，Confidentiality 机密性**——车辆数据不能被未授权访问。比如车主的导航历史、车内通话记录，这些不能让别人看到。
- **I，Integrity 完整性**——车辆系统和数据不能被篡改。比如刹车 ECU 的固件，不能让人偷偷刷个恶意版本进去。
- **A，Availability 可用性**——车辆系统要能正常运行不中断。比如有人发起 DoS 攻击让你的雷达瘫痪，这就破坏了可用性。

AAA 是后来补充的三个原则，更偏工程落地：

- **Authentication 认证**——你是合法的吗？比如 ECU 互相通信时要先确认对方身份。
- **Authorization 授权**——就算你是合法的，你有权限做这件事吗？比如普通 App 不能去改防抱死系统的参数。
- **Accountability 可追溯**——做过什么事都要能查到、不能抵赖。出了事能定位到具体是谁、什么时候、改了什么。

这六个原则在后面**几乎每一页技术内容都会用到。**后面我会讲 STRIDE 威胁模型，那六种威胁正好对应这六个原则的反面——攻击者就是来破坏这六性的。⏸️

现代汽车的智能化特征（原 slide 6，约 2 分钟）

那为什么现代汽车的安全这么难做？因为现在的车已经不是十年前那种"机械主导、电子辅助"的形态了。我们一起看看现在一台车上有多少个"门"是连通外面的。

第一个，**车载信息娱乐系统。**大屏、语音、导航、应用商店——这就是个安卓平板，平板有的漏洞它都有。

第二个，**自动驾驶系统。**一堆传感器、AI 决策、还要 OTA 升级。任何一个环节被篡改，决策就可能出错。

第三个，**车联网 V2X。**车跟车通信、车跟基础设施通信，5G 全部接入。这意味着车直接暴露在公网上。

第四个，**移动应用集成。**手机 App 能远程控制车辆、能解锁、能启动。手机被入侵，车也跟着被入侵。

第五个，**OTA 在线升级。**云端可以直接更新车上的固件——这是双刃剑，能修漏洞，也能被攻击者用来推恶意版本。

第六个，**电池管理系统。**BMS 看似不起眼，但它涉及到充电、热管理、续航策略，一旦被恶意控制，电池热失控可不是开玩笑的。

大家数一下，光这一页就六个攻击面。每个攻击面背后可能又有几个、十几个细分的入口。这就是为什么汽车信息安全比传统 IT 安全要难得多——攻击面太广，而且**很多入口是无线的、远程的。**

为什么如此重要（原 slide 7，约 2 分钟）

讲完攻击面，再讲讲一旦被攻破后果有多严重。我们从三个维度看。

第一，安全维度——会死人的。

- 人身安全：黑客远程接管车辆，刹车失效、转向失控，这是真的会出人命的。
- 财产安全：车辆被盗、被恶意损坏。
- 公共安全：如果攻击者批量控制几万辆车在同一时刻刹车，城市交通就瘫痪了。

第二，隐私维度——你的生活被人看光。

- 位置隐私：每天去哪、几点出门、几点回家，这些数据泄露了，等于把你的生活作息送给陌生人。
- 行为隐私：开车习惯，激进还是温和，能反推出你的性格甚至健康状况。
- 通信隐私：车内蓝牙电话、语音指令，全都可能被截获。

第三，经济维度——企业和用户都赔钱。

- 企业成本：召回、品牌损失、法律赔偿，单次事件赔几亿都很常见。
- 用户损失：车被盗、个人信息泄露的直接损失。
- 合规成本：法规跟不上，市场就关门。

最右边两个数字大家一定要记住：**2023 年全球汽车网络安全事件增长了 99%**，几乎是翻倍。**全球主要市场已经强制要求**——不合规的新车禁止上市。所以信息安全已经不是"加分项"，是"准入项"。

真实案例（原 slide 8，约 2.5 分钟）

光讲数字大家可能没感觉，我挑四个真实案例讲一下，让大家有具体的画面感。

第一个，2015 年 Jeep Cherokee 事件。两个安全研究人员，远在十几公里外，通过车载娱乐系统的漏洞，**远程接管了一辆正在高速公路上行驶的 Jeep**——不是停车后控制，是行驶中控制刹车、转向、加速。这个事件直接导致 FCA **召回 140 万辆车**，赔了一个多亿美金。这是汽车信息安全行业的"分水岭"事件，从那以后，所有 OEM 才真正开始把网络安全当回事。

第二个，2019 年欧洲多国的智能钥匙中继攻击。攻击者用一个简单的中继设备，**在你家门口接收你钥匙发出的信号，放大后转发到你停在街边的车旁边**——车以为钥匙就在身边，就开锁、启动了。多个豪华品牌中招，保险赔付**超过 3000 万欧元。**这件事的教训是——只要是无线通信，就必须加密 + 距离绑定，不能光靠"信号到了"就认证通过。

第三个，2021 年某新能源车企数据泄露。50 万用户的隐私数据被脱库——包括车辆轨迹、个人信息。罚款 + 集体诉讼，损失非常惨重。教训是：车端安全做得好不够，云端服务器、API 接口同样要严防死守。

第四个，2022 年欧洲充电桩攻击。攻击者发现充电桩管理网络的漏洞，**远程让数万个充电桩同时停止充电。**电动车主在路上趴窝。这告诉我们——**车联网生态系统的安全是整体的**，光自己的车安全没用，配套的充电桩、地图、TSP 后台都要安全。

⏸️ 这四个案例横跨七年，从车辆本身到钥匙、到云端、到充电桩——**攻击面只会越来越广，从来不会变窄。**

常见的攻击方式（原 slide 9，约 2 分钟）

把上面四个案例的攻击入口归类一下，就形成了这页的四大攻击面：**云端、通信、车内、终端。**

云端 Cloud——OTA 服务器、TSP 后台、API 接口。攻击者攻破云端，可以批量推恶意固件，影响面最大。

通信 Comm——蜂窝 4G/5G、V2X、Wi-Fi、蓝牙。这是车辆与外界连接的桥梁，桥被人控制了，进出都不安全。

车内 In-Vehicle——CAN/CAN-FD 总线、车载以太网、OBD-II 诊断口。**注意，我们的角雷达 ECU 就在这一层**，是车内总线上的一个节点。诊断口尤其要注意，4S 店一插就能读写，物理隔离一旦被绕过就完了。

终端 Endpoint——TBox 网关、智能钥匙、充电口。这些是攻击者最容易物理接触到的入口。

我特别在车内层标了一颗星——**雷达 ECU**，这就是我们今天第三章的主角。**整个第三章我们就是在围绕这一颗星，讲它怎么从硬件到软件、从启动到运行、从工厂到报废，全方位做安全。**

第二部分 · 信息安全核心技术（约 17 分钟）

进入第二章（slide 10，约 5 秒）

第一章给大家铺好了"为什么"和"防什么"的认知。第二章开始讲**业界用什么武器来防**——这一章是通用的核心技术，不局限于汽车。

加密技术（原 slide 11，约 2 分钟）

最基础的武器就是**加密**。加密分两大流派：对称加密和非对称加密。

对称加密，代表就是 **AES**——高级加密标准。一把钥匙，加密和解密用同一把。优点是**快**，硬件加速器一秒能跑几个 GB；缺点是**密钥怎么安全分发**是个难题——你得想办法把这把钥匙安全地告诉对方，但你还没建立安全通道呢。所以 AES 主要用在**已经建立信任之后的大数据量场景**——固件加密、日志保护、数据库加密、通信加密都用它。

非对称加密，代表是 **RSA 和 ECC**。一对钥匙，公钥加密只能私钥解，反过来也行。好处是**不需要预先分发密钥**——把公钥到处贴都没事，私钥自己留着就行。缺点是**慢**，比 AES 慢上百倍甚至上千倍。所以非对称加密**不会用来加密大数据**，主要用在**身份认证、数字签名、密钥交换、证书验证**这种小数据量但要求高安全的场景。

实际工程里两种是混合用的：用非对称加密**先安全交换一把对称密钥**，然后真正传数据用对称加密——这就是 TLS 的基本套路。

最下面这条警告大家一定要注意：**NIST 已经宣布 RSA-2048 在 2030 年后弃用**。后量子密码 ML-KEM、ML-DSA 即将进入车规标准。我们做车规芯片，**生命周期动辄 10 年以上**，现在新设计就要考虑 PQC 兼容性，否则 2030 年一到，老芯片立刻就过时了。

完整性验证（原 slide 12，约 2 分钟）

加密解决"别人看不到"的问题，**完整性验证**解决"别人改不了"的问题。这页三种工具，强度从弱到强。

第一个，Hash 哈希。代表算法 SHA-256、SHA-512、国密 SM3。**特点是防误传，但不防伪造。**你下载一个文件，比对一下 hash 看下载有没有错——这是 Hash 的典型用法。但 Hash 完全公开，**任何人都能算**，所以攻击者改了文件，重新算个 hash 给你，你照样会通过。Hash 本身没有"身份"概念。

第二个，HMAC 消息认证码。在 Hash 基础上**加一把共享密钥**——只有知道这把密钥的人才能算出正确的 MAC。代表算法是 HMAC-SHA256、CMAC-AES。这就**能防伪造了**——因为攻击者没有密钥，算不出正确的 MAC。**典型应用：CAN 总线和车载以太网的帧认证（也就是 SecOC）**——每一帧 CAN 报文都带上 MAC，伪造的报文一发就被识破。但 HMAC 有个问题：**对称密钥要双方都有，大规模分发难**——几百万辆车每辆都要分发密钥，怎么管理是个大工程。

第三个，数字签名。用非对称加密做的认证——签名用私钥，验证用公钥。代表算法 RSA-PSS、ECDSA、Ed25519。这个**最强**，既防伪造又**不可抵赖**——因为只有你有私钥，别人验证签名通过了，你赖不掉。**典型应用：镜像签名、X.509 证书、OTA 包认证。**缺点也很明显——**比 HMAC 慢 100 到 1000 倍**，不可能每帧 CAN 都用数字签名，那总线就堵死了。

所以工程上是分层用的：每帧 CAN 用 HMAC（SecOC），整个固件镜像用数字签名，文件校验用 Hash。**强度和性能要平衡。**

PKI 与 X.509 证书体系（原 slide 13，约 2.5 分钟）

刚才讲数字签名的时候，关键问题是——**我怎么知道这个公钥真的是对方的？** 万一中间人换了公钥呢？答案是 **PKI——公钥基础设施。**

PKI 的核心思想是**信任链 Chain of Trust。**我们看左边这个三层结构：

最顶上是 **Root CA 根证书**——**自签名的，离线保存**，是整个信任体系的"万年信任之锚"。它的私钥只在物理金库里，用一次拿出来一次。

中间是 **Intermediate CA 中间证书**——由 Root CA 签发，**可以在线、可以撤销。**它的作用是日常签发设备证书——这样万一中间 CA 被攻破，撤销它就行，不用动 Root CA。

最下面是 **Device Cert 设备证书**——比如我们 ECU 的证书，由 Intermediate CA 签发，**最终烧到芯片的 eFuse 里。**

验证的时候反向走：拿到设备证书，**用 Intermediate CA 公钥验它的签名**；要验 Intermediate CA，就**用 Root CA 公钥验**；Root CA 是自签名的，被预先信任。这就是"信任链"——任何一环断了，整个链都不可信。

右边是 **X.509 证书的标准结构。**我们看几个重要字段：

- **Version**：现在通用 v3
- **Serial**：唯一序列号，撤销时按这个号
- **Issuer**：谁签发的
- **Validity**：有效期
- **Subject**：证书持有者
- **Public Key**：核心——里面装的就是持有者的公钥
- **Extensions**：扩展字段，**这是后面要重点讲的**——TI 就是用 Extensions 里塞自定义 OID 字段，把 Boot 信息塞进去
- **Signature**：Issuer 用私钥对前面所有字段的签名

记住右下角这一行："**TI 用 Extensions 里塞自定义 OID 字段保存 boot 信息**"——这是后面 slide 30 到 32 讲的核心机制，**TI 是怎么把 Secure Boot 所需的元数据，通过 X.509 证书的天然签名机制保护起来。**

安全启动与信任执行（原 slide 14，约 2 分钟）

有了加密、签名、证书这些基础工具，组合起来就能实现**安全启动 Secure Boot**——这是 ECU 安全的基石。

我们先看左边的**启动信任链**，这是一个**逐级验签**的过程：

Boot ROM——固化在芯片的硬件 ROM 里，**不可修改**，是整个信任链的"硬件根信任"。它做的第一件事就是验证下一级 Bootloader 的签名。

第一阶段 Bootloader——被 Boot ROM 验证通过后才能跑，初始化基础硬件，加载下一阶段。

第二阶段 Bootloader——验证操作系统的完整性和真实性。

操作系统内核——验证应用程序的签名。

这就是"链式信任"——上一级被信任的，去信任下一级；下一级先被验证，再被加载执行。**任何一环验证失败，整个链停下来。**

中间是**可信执行环境 TEE**——硬件层面把整个芯片分成"安全世界"和"普通世界"，敏感的密钥操作只在安全世界里跑，普通世界根本碰不到。**安全存储、安全运算、安全通信**全部在 TEE 里完成。

右边是**数字签名验证**的几个关键点：用 RSA 或 ECC 公钥验签、走完整证书链、还要做**防回滚保护**——防止有人故意刷一个旧版本的固件来利用旧漏洞。这个防回滚后面在讲 SWRV 字段时会详细展开。

安全生命周期管理（原 slide 15，约 1.5 分钟）

刚才讲的都是"运行时"的安全，但安全不是只在车跑起来才需要的——**它是一辆车从概念到报废全生命周期的事。**

整个生命周期分六个阶段：**概念→设计→测试→生产→运营→报废**。每个阶段都有相应的安全活动。

中间这个**成本对比是关键**——**设计阶段修复一个安全问题成本是 1 美元，到生产阶段是 1000 美元，召回阶段是 10000 美元**——三个数量级的差异！这就是为什么我们一直强调"**安全要前移**"，要在概念和设计阶段就介入，不要等到测试阶段才发现问题。

右边"**持续改进**"也很重要——安全不是一次性认证完就完事的。**漏洞每天都在被发现**，必须有持续的威胁情报收集、漏洞管理、应急响应演练。我们做完一个项目之后还要持续监控这个产品在野的安全状态，必要时通过 OTA 推送补丁。

安全测试方法（原 slide 16，约 2 分钟）

讲完生命周期的概念，看具体的测试手段——**业界主流的六种安全测试方法**：

SAST 静态分析——源码级看缓冲区溢出、未初始化变量、危险 API。**Code Review 阶段做**，我们常用的工具是 Coverity、Polyspace。这是最便宜的发现 bug 的方式。

DAST 动态测试——黑盒测试，运行时构造异常输入观察行为。**集成测试阶段做。**

Fuzzing 模糊测试——这个我特别推荐——**随机或者智能变异的方式造大量奇怪输入**，找崩溃、内存越界、协议解析漏洞。**持续集成阶段长跑**——CI 上挂一个 fuzzer 24 小时不停跑，能挖出大量人工想不到的边界 bug。

PenTest 渗透测试——白帽以攻击者视角端到端验证。**型式认证之前必做。**一般请第三方专业团队，因为自己人有思维定势。

HW Side-Channel 侧信道攻击——测功耗、电磁辐射、时间——这是攻击者通过物理观测来猜密钥。HSM 模块和加密硬件**必须单独做这个测试**。工具是 ChipWhisperer、Riscure。

Fault Injection 故障注入——电压毛刺、激光、时钟干扰——目的是让芯片在异常情况下"出错"，从而绕过 Secure Boot。这是 Tier-1 实验室才有的能力。

这六种不是二选一，是组合用的——开发阶段 SAST+Fuzzing，集成阶段 DAST，发布前 PenTest+侧信道+故障注入。**层层过滤、层层补强。**

国际标准与法规（原 slide 17，约 2.5 分钟）

技术讲完，看法规——这是**强制性的、不合规就不能上市**的内容。

左边三个**强制法规**：

UN R155——网络安全管理体系 CSMS。要求 OEM 建立从研发到报废**全生命周期**的网络安全管理流程，包括风险评估、监测、响应。**这是欧盟的入场券。**

UN R156——软件更新管理体系 SUMS。要求 OEM 建立**可追溯、可回滚**的 OTA 流程，**每次升级都要通过型式认证**——大家想想这意味着什么，OTA 不是想推就能推的。

GB 44495——中国国标，2024 年发布、2026 年 1 月强制实施——也就是说现在已经强制了。**对标 R155，但增加了国密 SM 算法的要求**。所以我们做国内市场的项目，必须支持 SM2、SM3、SM4。

右边三个**技术参考标准**——它们是法规落地的具体抓手：

ISO/SAE 21434:2021——这是 R155 落地的"圣经"，定义了 **TARA 威胁分析、CAL 网络安全保证等级、所有安全活动**。基本上所有 OEM 内部流程都映射到 21434。

ISO 24089:2023——R156 的技术指南，定义 SuMS 流程、Update Package 结构、**回滚机制**。

ISO 21177 / 26262——21177 管 V2X 通信信任；26262 大家更熟悉，是**功能安全**——和信息安全必须并行考虑。**功能安全 + 网络安全 + SOTIF**，这三个是现在车规开发的"三位一体"，缺一不可。

🟦 **法规层面的核心信息**：现在已经不是"要不要做"信息安全的问题，是**强制做的同时怎么做得高效合规**。

第三部分 · 角雷达信息安全（约 26 分钟）

进入第三章（slide 18，约 5 秒）

好，前面铺了 30 分钟，终于到大家最关心的——**我们手上的角雷达 ECU 怎么落地信息安全**。

角雷达的位置和威胁（原 slide 19，约 2 分钟）

我们先定位角雷达在整车里的角色。看左边这个示意图——一辆车四个角各有一个雷达：**前左 FL、前右 FR、后左 RL、后右 RR**，全部连到 ADAS 域控制器上。它们的数据**直接喂给自动驾驶决策**——所以一旦数据出错，决策就会出错，**轻则误报、重则撞车**。

右边是经典的 **STRIDE 威胁分析模型**——这是微软提出的、业界通用的威胁建模方法，正好对应六类攻击：

- **S Spoofing 假冒**——攻击者伪造一个雷达 ECU，**上报虚假障碍**给域控制器，让自动驾驶以为前方有障碍紧急刹车。

- **T Tampering 篡改**——刷入恶意固件，**屏蔽真实障碍**，让自动驾驶看不到本该看到的危险。
- **R Repudiation 抵赖**——事故后否认数据来源——这在事故责任认定时是大问题。
- **I Info Disclosure 信息泄露**——导出我们辛辛苦苦标定的参数，**算法 IP 被竞争对手拿走。**
- **D DoS 拒绝服务**——高频干扰让总线饱和，**雷达彻底失效。**
- **E Elevation of Privilege 提权**——通过诊断口提权，刷写本不该被刷写的区域。

⏸ **大家想想——这六类威胁，前面 STRIDE 这六个字母，是不是正好对应 CIA + AAA 那六个原则的反面？** S 对应 Authentication 被破坏，T 对应 Integrity 被破坏，R 对应 Accountability 被破坏，I 对应 Confidentiality 被破坏，D 对应 Availability 被破坏，E 对应 Authorization 被破坏。这就是 STRIDE 模型为什么这么经典——**它和安全原则是一一对应的**，攻击和防御天然成对。

STRIDE → CIA → 工程对策（原 slide 20，约 2 分钟）

接着上一页的逻辑，这页就是把"威胁→属性→对策→落地硬件能力"全部串起来——**这是这次培训最重要的一张表**，希望大家拍下来反复看。

我从上到下过一遍：

Spoofing 假冒——破坏 Authentication——对策是**证书加 ECU 唯一身份**，每颗芯片都有 TIFEK——TI Factory Embedded Key——硬件层面的全球唯一身份。落地能力是 **X.509 证书 + HSM 硬件密钥。**

Tampering 篡改——破坏 Integrity——对策是 **Secure Boot 加镜像签名**——后面会展开。落地能力是**验签链。**

Repudiation 抵赖——破坏 Accountability——对策是**签名日志 + 时间戳**，所有关键操作都有不可抵赖的记录。落地能力是 **HSM 内的安全计时器。**

Info Disclosure 信息泄露——破坏 Confidentiality——对策是**镜像加密 + 调试口加锁。**落地能力是 **AES + WIR Mode 锁**——WIR 是 TI 的设备生命周期状态，后面会讲。

DoS 拒绝服务——破坏 Availability——对策是 **SecOC 帧认证 + 看门狗**——SecOC 就是前面 HMAC 那一页讲的 CAN 帧认证。落地能力是 **HSM 加速**——因为每帧都算 MAC，没有硬件加速根本扛不住。

Elevation of Privilege 提权——破坏 Authorization——对策是 **UDS 0x29 服务 + 角色控制**——UDS 0x29 是诊断协议里的认证服务。落地能力是 **HSM Key Slots。**

⏸ **这张表的意义在哪里？** 它告诉我们——**每一个工程对策不是凭空想出来的，背后都对应一个具体的威胁和一个被破坏的属性。**我们做安全设计的时候，**要从威胁出发，倒推到属性，再倒推到对策**——而不是看到一个酷炫的技术就盲目往项目里塞。

雷达 ECU 整体安全方案（原 slide 21，约 2 分钟）

把上一页的对策落到我们雷达 ECU 上，整体方案分**三层**：

最底层是硬件根 Hardware Root——这是一切的基础。**TI HSM 是一颗 Cortex-M4 的安全协处理器**，跑在主芯片旁边专门处理安全任务。**eFuse OTP** 存放根密钥——一次性可编程，烧进去就改不了。**TRNG 真随机数发生器**保证密钥生成的不可预测性。**DTHE 加解密硬件加速器**保证 AES、Hash 这些运算的高性能。

中间层是启动安全 Secure Boot——HSM ROM 验证最早的代码，然后逐级验签：**HSM ROM 验签** → **R5 SBL 验签** → **App 验签**——这就是前面讲的"信任链"。最后还有**反回滚 SWRV**——防止刷旧版本。

最上层是运行时安全 Runtime——也就是车在路上跑的时候。**SecOC 帧认证**保护 CAN 报文，**Secure Storage** 保护敏感数据存储，**Secure Log** 保证日志不可抵赖，**TLS Channel** 保护对外通信。

ℹ️ 大家注意这是"**洋葱模型**"——**最外层是运行时，往里一层是启动安全，最里层是硬件根**。攻击者要攻破最外层 Runtime，可能还能通过 OTA 修复；但如果攻破到硬件根，**整颗芯片就废了**——所以越往里安全要求越高，硬件根是不能妥协的。

AWR2x44 芯片安全架构（原 slide 22，约 2 分钟）

具体到我们用的 TI AWR2x44 芯片，它提供了六大安全特性：

Secure Boot 安全启动——确保只有经过验证的软件才能运行，链式信任。

HSM 硬件加密——**Cortex-M4 的硬件加密协处理器**，独立于主 CPU，攻击者就算控制了主 CPU 也碰不到 HSM 内部的密钥。

Secure OTP 安全存储——**一次性可编程存储器和 eFUSE**，存密钥和敏感数据。烧进去就改不了，这是它的优点（防篡改）也是缺点（错了改不掉，所以**烧 eFuse 必须万分谨慎**）。

JTAG Lockout 调试保护——量产时锁死 JTAG，防止逆向工程。后面 Secure Debug 那页会详细讲怎么在锁死的状态下临时打开调试。

Secure Flash 安全固件更新——OTA 升级时也要走完整的签名验证。

Access Control 内存隔离——不同应用之间内存隔离，防止互相串扰。

这六个特性是 TI 给我们的"工具箱"，但工具箱不会自动保护我们——**用得好不好，看我们的设计**。后面几页就是讲怎么把这些工具用对。

HSM 两层概念澄清（原 slide 38，调整位置，约 2 分钟）

【重要：这一页是新调整位置过来的，原本在 37 之后。这是非常关键的概念澄清，必须在介绍 HSM 软件栈之前先讲，否则后面"HSM 软件栈"和"TI HSM 硬件架构"听众会混乱。】

这页要花点时间——**这是整个第三章最容易让人混乱的地方**。我们说的"HSM"在不同上下文里**有两个层次的含义**，必须分清楚。

Layer 1：TI HSM ROM——芯片硬件层

- 这是 **TI 固化在 Mask ROM 里的代码**，不可改、不能改。
- **职责**：Secure Boot、密钥管理、X.509 验签、DTHE 加解密。
- **状态机**：**GP** → **HS-FS** → **HS-SE**，这是 TI 设备生命周期，由 eFuse 控制——后面 Slide 25 会详细展开。
- **密钥域**：TIFEK、BMPK/BMEK、SMPK/SMEK、MPK/MEK、KEK——一堆缩写后面慢慢讲。
- **服务对象**：**整颗 SoC**——R5F-0、R5F-1、DSP、HSM-RT 全都受它管。

Layer 2：Bosch HSM-RT——应用层

- 这是**博世自己开发的、跑在 TI HSM 那个 Cortex-M4F 上的固件**——**客户可定制**。
- **职责**：应用层加密服务、安全存储、密钥派生、CAN-Sec/SecOC、自定义生命周期。
- **状态机**：**Init** → **Field** → **Compromised** → **Decom**——这是博世自定义的应用生命周期，**用 RAM/Flash 里的标志位实现**。
- **密钥域**：OEM 业务密钥——CAN-MAC Key、OTA Session Key、数据签名密钥。

- **服务对象**：**R5F-0/1 上的应用代码**，通过 IPC + Mailbox 调用。

关键关系：HSM-RT 是 TI HSM 上跑的"应用程序"，由 boot_seq=0 加载——**也就是说 TI HSM ROM 先启动，然后加载并验证 HSM-RT，HSM-RT 启动后才开始为业务提供服务。**

⏸️ **大家一定要记住——后面我们说"HSM 做了什么"，要先问一句：这是 TI HSM ROM 做的，还是 Bosch HSM-RT 做的？** 这两层职责不同、密钥不同、生命周期不同，混淆了排查问题就乱套。

HSM 软件栈（原 slide 23，约 1.5 分钟）

接着上一页的两层概念，看 HSM 软件栈。

整个软件栈最上面是**客户应用 Customer App**，跑在主核 R5F 上。客户应用通过两套接口调用 HSM 的功能——**AutoSAR CSM Interface** 和 **PKCS#11 Interface**——前者是车规标准接口，后者是通用的密码学接口。

中间是**博世的 HSM-RT 固件**——也就是 Layer 2 那一层——里面有安全应用、密钥管理、加密原语、安全启动模块、底层驱动。

最底下是**硬件层**——HSM 协处理器、DTHE 加密硬件、Secure RAM、HSM ROM、eFuse、TRNG。

⏸️ 大家可以这样理解：**应用通过标准 API 调用 HSM 服务，HSM-RT 把这些请求翻译成对硬件的操作**——客户应用不需要直接碰硬件。

TI HSM 硬件架构（原 slide 24，约 1.5 分钟）

这页是 TI HSM 的硬件框图，我重点讲红框里的几个核心模块：

Cortex-M4F 内核——HSM 的"大脑"，跑 HSM ROM 和 HSM-RT 固件。它是**完全独立于主芯片其他核**的，主核被攻破也碰不到它。

Secure ROM——固化的 TI HSM 代码，从这里开始执行。

DTHE——硬件加密加速器，AES、SHA、PKA（公钥加速）、TRNG 都在这里。**没有 DTHE 的话，靠 M4F 软件做密码学运算根本扛不住实时性。**

eFuse / OTP——根密钥的物理存储位置。

Secure RAM——只有 HSM 能访问的 RAM，主核读不到。

右侧的接口 和主芯片的 IPC、Mailbox 通信——主核要做加密就发个消息给 HSM，HSM 做完结果再返回。

⏸️ 这个硬件隔离是**物理级别的**——不是靠软件权限分隔，而是芯片走线就隔开了。**这是 HSM 安全等级远高于普通 MCU 的根本原因。**

Secure Device Lifecycle（原 slide 25，约 2 分钟）

接下来三页讲生命周期——**【Slide 41 也调整到这里讲】**。

我们看 TI 设备的生命周期——**SoC 一辈子要经过四个阶段**：

Phase 1：No Root——**裸芯片**，没有任何客户密钥，TI 出厂时的状态。Type 是 GP 或 HS-FS。

Phase 2：TI Keys Only——TI 写入了自己的根密钥 TIFEK，但还没有客户密钥。Type 仍是 HS-FS。**这个阶段芯片可以被任何客户拿去用**——属于"通用"状态。

Phase 3：Customer Provisioning——关键阶段。客户在工厂里把自己的根密钥（BMPK 公钥的 hash）烧到芯片的 eFuse 里，**同时把芯片的 lifecycle state 推到 HS-SE。**从此这颗芯片就和客户绑定了，**只能加载客户私钥签的镜像。**

Phase 4：Deployment in Field——这就是车上跑的状态。Type 是 HS-SE，所有安全机制全部启用。

⏸️ **这个生命周期是单向的，不能往回退。**也就是说从 GP → HS-FS → HS-SE，烧 eFuse 的动作是不可逆的——**这就是为什么 OTP Provisioning 的工厂工序必须做到极致严谨**，烧错一颗就废一颗。

HS→SE Solution / OPT Writer（原 slide 26，约 2 分钟）

这页讲**最关键的工序——客户密钥怎么注入芯片。**

整个流程分两个域：左边是 **Controlled Delivery（HS-FS Device）**，右边是 **Customer Development and Production（HS-SE device）**。

左边的流程：

- **Bosch Device Programmer** 负责给芯片写入客户的 OTP 配置——**包括 BMPK 公钥的 hash、芯片的 lifecycle 状态、各种安全配置。**
- 这个 Programmer 工具就是俗称的 "**OPT Writer**"。
- 写入的内容来源于 KMS 服务器——**根密钥从 KMS 来，经过 PKS 分发，最后由 OPT Writer 烧到芯片。**

右边的流程：

- 芯片烧完后状态是 HS-SE，**进入正常的客户开发和量产流程。**
- 应用代码、HSM-RT 代码全部用客户的根密钥签名后，烧进 Flash。

⏸️ **OPT Writer 这道工序是整个安全体系的"信任之锚"**——这一步如果出问题，比如密钥被偷、被篡改、烧错——后面再多的安全机制都救不回来。所以**这道工序必须有专门的安全产线、专人值守、全程审计。**

HSM Lifecycle（原 slide 41，调整位置，约 2 分钟）

【重要：这一页是新调整位置过来的，原本在第三章末尾。这里和上面 SoC lifecycle 一起讲，逻辑更连贯——一个讲 SoC 层生命周期，一个讲 HSM 应用层生命周期。】

刚才讲的是 SoC 的生命周期，**HSM-RT 应用层自己也有一套生命周期**——这就是前面 38 页提到的 "Layer 2 状态机"。看这张表：

HSM LC = NA (0xFF) ——SoC 状态 HS-FS——含义是 HSMRT 还没启动。

HSM LC = Invalid (0) ——SoC 状态从 HS-FS → HS-SE 转换中——板载 Flash 使能 OTP，FS 变 SE。

HSM LC = Manufacturing (1) ——SoC HS-SE——**工厂状态。**

HSM LC = Development (4) ——SoC HS-SE——**JTAG unlocked，无需授权即可调试**——开发阶段用的。

HSM LC = Operation (2) ——SoC HS-SE——**JTAG locked，量产状态**——这是车上跑的最终状态。

HSM LC = Return (3) ——SoC HS-SE——返修状态。

HSM LC = Decommission (5) ——SoC HS-SE——报废状态，密钥被销毁。

下面有个 Sample 例子——**C0 是 FS、C1 是 LC Dev、D 是 LC Series（量产）**——大家在烧片日志里看到这些标识就知道在哪个状态。

ℹ️ 注意右边那张状态机示意图——**Manufacturing → Development → Operation 是正常路径**，**Operation → Return → Decommission 是返修和报废路径。HSM-RT 不是单向的**，可以从 Operation 退回 Return 做返修——但这有严格的认证流程，不是想退就能退。

密钥管理 KMS / PKS（原 slide 28，调整位置，约 2 分钟）

【重要：原来 slide 28 在 slide 27 之后，现在调整为先讲 28（管理体系），再讲 27（撤销机制），符合"先框架后特例"的逻辑。】

刚才几页都提到了"密钥从哪来"——这页就讲整个密钥管理体系。我们博世的密钥管理分两层：

KMS——Key Management System

- 服务器**在博世德国总部**，最高安全等级。
- 主要负责**密钥的生命周期管理**——生成、分发、吊销、销毁。
- **所有的根密钥都来源于此**——BMPK、BMEK 这些"宇宙级"密钥都是 KMS 生成并保管的。
- 它还作为**后端服务响应来自 PKS 的密钥请求**——产线要密钥就向 PKS 申请，PKS 再向 KMS 申请。

PKS——Production Key Server

- 这是**面向工厂的执行前端**——产线连接的都是 PKS，不直接连 KMS。
- 职责：**从 KMS 获取密钥并注入到设备中。**
- 通过 OPT Writer 把密钥烧到芯片的 eFuse 里。

ℹ️ **为什么要分两层？** 因为根密钥太宝贵了，**根密钥不能直接暴露在工厂网络里**——工厂网络再严格也是生产环境，比 KMS 那种"金库级"环境差得远。所以中间加一层 PKS 做隔离——**PKS 只缓存当前工厂任务需要的密钥，用完就清。**

整个层次是：**KMS（金库）→ PKS（前置仓）→ OPT Writer（执行器）→ 芯片 eFuse（最终目的地）**。

根密钥撤销（原 slide 27，调整位置，约 2.5 分钟）

【重要：调整后先讲了密钥管理（28），现在讲撤销（27）就有了铺垫——撤销是密钥管理的一个特殊场景。】

讲了 KMS/PKS 的密钥管理体系，**最坏的情况怎么办？根密钥泄露了怎么办？** 这就是 Key Revision——根密钥撤销机制。

看左边的状态机，**只有两个状态**：

- **State 0（出厂默认）**：使用主密钥 BMPK + BMEK
- **State 1（撤销已生效）**：切换到备份密钥 BMPK#1 + BMEK#1

触发的方式是 eFuse 烧 1 个 bit——One-Way 单向不可逆。

触发场景有三种（看右上角）：

1. **签名/加密私钥意外泄露**——员工流失、签名工具被入侵。
2. **算法被时代淘汰**——比如 RSA-2048 安全等级不够了，要升级到 RSA-3072 甚至 4096。
3. **OEM 安全策略变更**——主动换签名密钥。

右下角的撤销操作流程是标准动作，6 步：

1. **准备新密钥包**——用 BMPK#1/BMEK#1 重新签名加密镜像。
2. **OTA 推送过渡版本**——确保所有车端都先收到新签名镜像。
3. **确认 100% 安装——这一步极其关键，没安装的车一旦执行第 4 步就启动不了。**
4. **执行 eFuse 烧写**——Key Revision 0→1，**不可逆。**
5. **ROM 自动切换**——下次启动用 BMPK#1 Hash 验签。
6. **停用主密钥**——服务端撤销主签名密钥。

ℹ️ **这是一个高风险动作**——一旦烧错或者步骤 3 没做到 100%，**整批车直接变砖。**所以业界的真实做法是——**根密钥撤销几乎从来不主动用**，是兜底机制。一旦用了，意味着发生了重大安全事件。

X.509 证书的生成（原 slide 29，约 2 分钟）

回到技术细节——前面 slide 13 讲了 X.509 证书的概念，这页讲它在我们项目里**具体是怎么生成的——用 OpenSSL 三步搞定。**

第一步：生成密钥对

```
openssl genrsa -out priv.pem 3072
openssl rsa -in priv.pem -pubout -out pub.pem
```

生成 RSA-3072 私钥，然后导出公钥。**红字警告——私钥要严密保管，存 HSM**——私钥一旦泄露整个链就完了。

第二步：构造证书模板
写一个 OpenSSL 配置文件，里面定义证书的字段——CN、Issuer 这些标准字段，以及**最关键的——TI 自定义 OID 字段。**
看模板里的 `1.3.6.1.4.1.294.1.1` 和 `1.3.6.1.4.1.294.1.2`——**这两个就是 TI 自定义的 OID，分别塞 boot_seq 和 image_integrity**——下一页详细讲。

第三步：签发自签名证书

```
openssl req -new -x509 -key priv.pem -out cert.pem -days 7300 -config cert.cfg
openssl x509 -in cert.pem -outform DER -out cert.der
```

用私钥对证书内容签名，生成 PEM 格式，再转成 DER 二进制——`cert.der` **就是最终拼接到镜像头的证书。**

ℹ️ 这个流程**完全可以在 CI 流水线里自动化**——后面 slide 37 会讲 CI/CD 集成，**关键约束是私钥必须留在签名服务器的 HSM 里，不出网。**

TI X.509 OID 扩展总表（原 slide 30，约 2 分钟）

接着上页讲——TI 在 X.509 证书的 Extensions 字段里塞了 **8 个自定义 OID**，把 Boot 元数据全部"附身"在证书里。

Base OID 是 `1.3.6.1.4.1.294.1.x`——这是 TI 在 IANA 注册的私有 OID 命名空间。

8 个字段一览：

- **.1 boot_seq**——镜像加载序号（HSM=0, R5 SBL=1, App=2, DSP=3）
- **.2 image_integrity**——hash 算法、hash 值、镜像长度、加载地址
- **.3 image_load_addr**——镜像在 SoC RAM 的加载物理地址
- **.4 swrv**——Software Revision，**反回滚比较用**
- **.5 encryption**——加密元数据，AES-CBC + IV + 密文长度
- **.6 key_derivation**——密钥派生模式
- **.7 debug**——Debug 控制，**JTAG 临时启用/禁用**
- **.8 core_id**——目标核——R5F-0、R5F-1、DSP、HSM 任选

底部那段橙色文字是**整个机制的精髓——为什么要把 boot 元数据塞到 X.509 里？** 因为 **X.509 证书的签名是对整个证书内容做的——OID 字段被一并 hash 和签名**——任何篡改都会导致验签失败。**这样我们就免费获得了 boot 元数据的完整性保护**——不需要再单独搞一套保护机制。

🟦 这是一个非常巧妙的设计——**借鸡生蛋**——借 X.509 这只成熟的"鸡"，下了我们 boot 元数据保护这颗"蛋"。

字段详解 · boot_seq + image_integrity（原 slide 31，约 2 分钟）

把刚才那 8 个字段中**最重要的两个**展开讲。

OID .1 · boot_seq——镜像加载序列号。
看左边那段配置：

```
INTEGER:0    ; HSM-RT 优先加载
INTEGER:1    ; 然后 R5F-0 SBL
INTEGER:2    ; R5F-1 应用
INTEGER:3    ; DSP 应用
```

ROM Code 严格按 boot_seq 升序逐一加载、验签、跳转执行。关键规则——如果 seq=0 的镜像验签失败，整链停止——不会跳过到 seq=1 继续。这就保证了**信任链的"单向逐级"性**——前面坏了后面就不跑。

OID .2 · image_integrity——镜像完整性元数据。
ASN.1 结构包含 5 个字段：

- `integrity_type`——hash 算法（1=SHA512, 2=SHA384）
- `integrity_value`——hash 值——**注意这是明文镜像的 hash**
- `image_size`——字节数
- `load_address`——物理加载地址
- `is_encrypted`——是否加密

右边讲 **ROM 验签的关键三步**：

1. 用 BMPK 公钥**验 X.509 整证书的签名**——通过后拿到可信的 `integrity_value`。
2. **DTHE 重新计算镜像的 SHA-512** —— 与 `integrity_value` 比较。

3. 一致 → 拷贝镜像到 `load_address` → 跳转。

⏸️ 注意右下角橙字——**加密镜像时，integrity_value 校验的是明文 Hash，即先解密再 hash**——而不是密文 hash。**这样保证了加密层和完整性层是解耦的，可以独立升级。**

字段详解 · swrv / encryption / key_derivation / debug（原 slide 32，约 2.5 分钟）

继续讲剩下四个字段——**这页有几个非常容易踩坑的地方。**

OID .3 · swrv (SoftWare Revision)——镜像版本号。
只有 5 个 bit，范围 0~31。每次新版本要 +1。ROM 验签后**对比当前 swrv 与 eFuse 中已记录的最高版本号**：

- new ≥ stored → 通过，**同时把 swrv 写回 eFuse**
- new < stored → **拒绝加载（Anti-Rollback 防回滚）**

陷阱：swrv 写 eFuse 是单调递增、**一旦烧高就回不去**——量产前必须严格管控版本号节奏。**5 bit 上限只有 31，烧到顶了整批芯片就再也不能升级了。**所以**量产首版常用 swrv=2 或 3，留低位空间给紧急回滚。**

OID .4 · encryption——加密元数据。

- `iv`：16 字节 AES IV，**每次构建随机生成**
- `encryption_algo`：1=AES-128-CBC, 2=AES-256-CBC

注意——**实际密钥 SMEK 不出现在证书里**！它由 ROM 用 KEK **现场派生**——见下一字段。

OID .5 · key_derivation——密钥派生。
TI 用 NIST SP 800-108 CMAC-KDF 派生 SMEK：

```
SMEK = KDF( KEK, salt || label || context, 256bit )
```

其中 KEK 是 **eFuse 中的 256 位根密钥，不可读，DTHE 内部消费——软件层永远拿不到 KEK 明文。**

关键安全实践：客户**每次出版本必须用新 salt——即使 KEK 不变，SMEK 也是一次性的。**这样攻击者破解了一个版本的 SMEK，也用不到其他版本上。

OID .6 · debug——授权调试矩阵。

- Bit0：JTAG 全开
- Bit1：仅授权特定 UID 设备开 JTAG
- Bit2：允许 secure ROM trace
- `debug_uid_check`：SoC Unique ID Hash

正确做法：**量产镜像 debug_perm_flags = 0x00（全锁）**，开发调试镜像绑定特定 UID——这样调试镜像就算泄露，也只能调试那一颗特定的开发板。

⏸️ **swrv 那个 5 bit 限制大家一定要记住**——这是真实事故的高发区。

Flash Layout（原 slide 39，调整位置，约 1.5 分钟）

【重要：原 slide 39 在这里上移，作为讲镜像创建（33）之前的硬件铺垫。】

讲镜像之前先看 Flash 布局——**镜像最终要存在 Flash 的哪些区域。**

这张表列出了所有的分区——每个分区有起始地址、大小、用途、加密属性、签名属性、所属核。

重点关注几个分区：

- **Boot 区**——存 HSM-RT 和 SBL，**必须签名验证。**
- **App 区**——存主应用代码——根据安全等级可能加密+签名。
- **数据区**——校准参数、配置、日志。
- **OTA 备份区**——升级时新版本先写到备份区，验证通过再切换——**这就是为什么 Flash 需要双区设计。**
- **Secure Storage 区**——只有 HSM 能访问，存证书、密钥、敏感数据。

⏸️ **Flash 布局是项目早期就要冻结的**——**改动一次代价巨大**——所有的工具链、签名脚本、OTA 包结构、回滚机制都依赖这个布局。**项目立项就要把 Flash Layout 表敲定。**

Block Define（原 slide 40，调整位置，约 1.5 分钟）

【和上一页一起讲，作为镜像创建的硬件铺垫。】

接着 Flash Layout，每个 Block 的具体定义——这页是 Flash 上每个块的详细字段定义，包括 block name、address、size、attribute、purpose 等。

重点告诉大家三件事：

1. **每个 block 都有明确的访问控制属性**——只有特定核或特定权限才能读/写。
2. **签名块和数据块是分开的**——签名值通常放在镜像头或专用区域，便于分别加载和验证。
3. **block 边界必须对齐到 Flash 的擦除单元**——通常是 64KB 或 256KB——**不对齐会导致 OTA 升级时无法独立擦写。**

⏸️ 大家在做新模块的时候，要根据这张表**提前规划好自己模块的 block 归属**——别等到代码都写完了发现没地方放。

镜像创建流程（原 slide 33，约 2 分钟）

铺垫完 Flash 布局，看镜像具体是怎么生成的。

整个流程可以分两条主线，对应**普通镜像**和**HSM 镜像**：

普通镜像（左侧）：

- 编译好的二进制 + 证书
- 经过签名工具
- 输出 "Image with certificate"

HSM Run Time 镜像（右侧）：

- HSM-RT 二进制
- 走另一条特殊的签名路径——**因为 HSM-RT 的签名密钥和普通应用不同**
- 输出"Encrypted Code"——通常 HSM-RT 是加密的

两路最终汇合——拼成一个完整的"可烧写镜像"，里面包含 Boot 区、App 区、HSM 区。

ℹ️ 大家注意**这个流程是签名流水线的核心**——后面 slide 37 讲 CI/CD 自动化，就是把这条流水线做到完全自动、不需要人工触摸私钥。

镜像验证流程（原 slide 35，调整位置，约 2 分钟）

【重要：调整后先讲创建（33）再讲验证（35），形成"单镜像的创建-验证闭环"，再讲整体启动序列。】

镜像生成后就要被验证——这页把验证流程拆得很细。

整个图分**三个区域**：

- **左边是"Image with certificate"**——也就是上一页生成的镜像，包含 X.509 证书 + 加密镜像。
- **中间是"Public Key + SMPK_Hash"**——验签所需的公钥，公钥的 hash 烧在 eFuse 里。
- **右边是验证后的输出"Binary Image"**——解密+验证通过后的明文镜像，可以加载执行。

整个验证流程：

1. 从镜像头**提取 X.509 证书**。
2. **计算证书内 Public Key 的 hash**，与 **eFuse 中存的 SMPK_Hash** 比较——一致才说明证书是客户的。
3. 用 Public Key **验 X.509 证书的签名**——一致才说明证书没被篡改。
4. 从证书的 OID 字段**提取 image_integrity 等元数据**。
5. 如果是加密镜像，**派生 SMEK 解密**。
6. **重新计算明文镜像的 hash**，与 OID 中的 integrity_value 比较——一致才能加载。

ℹ️ **整个过程涉及 hash 比对、签名验证、密钥派生、解密——全程在 HSM 内部完成，主核完全碰不到密钥**。这就是硬件根信任的精髓。

Secure Boot Sequence（原 slide 34，调整位置，约 2 分钟）

【重要：先讲单镜像验证（35），现在讲多镜像的整体启动序列（34）——递进逻辑。】

把上一页"单镜像验证"扩展到"整车启动"——这就是完整的 Secure Boot Sequence。

看这张时序图，**横轴是时间，纵轴是各个组件**：

HW Startup——通电瞬间，硬件先就位。

HSM ROM 启动——TI 固化的代码，**第一个被执行**——这就是"硬件根信任"。

HSM ROM 加载并验证 HSM RunTime——boot_seq=0 的镜像。

HSM RunTime 启动——开始为系统提供安全服务。

HSM RT 验证 R5 SBL——启动主核的 SBL。

R5 SBL 验证 BM——加载 Boot Manager。

BM 加载并验证 PFBoot 或 CustBoot——根据配置走不同的启动路径——**默认走 PFBoot，CustBoot 是客户自定义的。**

最后加载 App——这里 App 又分 CustApp 和 ProductionApp 两种。

关键标注：

- **"默认"路径**——量产时走的标准路径
- **"安全帧"**——传输过程要加签名/MAC 保护

ℹ️ **整条链是逐级验证、逐级跳转**——上一级被验证了才有资格验证下一级。**任何一级断了，后面全停**——这就是"链式信任"的强韧之处。

Secure Boot Process 详图（原 slide 42，调整位置，约 1.5 分钟）

【重要：紧跟上一页的 Secure Boot Sequence——两张图主题相同，连续讲不会让听众跳脱。】

刚才那张是**时序角度**，这张是**流程角度**——更详细地展开每一步在做什么。

我快速过一下关键步骤——大家会发现和上一页是同一个故事，只是视角不同：

1. **上电** → **HSM ROM 自检**
2. **HSM ROM 读 eFuse 获取 BMPK_Hash 和配置**
3. **HSM ROM 加载 HSM-RT 镜像** → **验签** → **跳转**
4. **HSM-RT 初始化** → **等待主核请求**
5. **R5F SBL 启动** → **调用 HSM-RT 验签自己**
6. **SBL 验签 App** → **跳转执行**
7. **App 启动** → **进入正常运行**

ℹ️ 上一页和这一页一起看，大家就能完整理解 Secure Boot——**时序 + 流程**，从两个角度认识同一个事情。**这就是为什么我把这两页放在一起讲，避免来回跳。**

Secure Debug & RMA Flow（原 slide 36，约 2.5 分钟）

讲完了"正常启动"，讲个反向场景——**车出问题了要调试怎么办？**

看左边的 **WIR Mode**——TI 设备的三种状态：

- **GP**：JTAG 全开——**原型开发用**
- **HS-FS**：JTAG 可控——试产用
- **HS-SE**：**JTAG 默认锁死**——量产用

问题来了——量产车坏了要回厂调试，JTAG 锁死的怎么办？ 答案是中间这个 **SECAP——Secure Debug Authentication Protocol**：

5 步认证流程：

1. **Debug 主机请求 JTAG**——表明想调试。
2. **HSM 发送 Challenge（随机数）**——挑战。
3. **主机用授权私钥签 Challenge + UID**——回应——**注意要带这颗芯片的 UID。**
4. **HSM 验签**——通过则解锁 JTAG。
5. **复位即失效**——下次需要重新认证。

关键安全特性：**授权私钥仅签该 SoC 的 UID —— 这意味着授权钥匙不可移植到其他芯片。**即使授权工具被偷了，也只能调试当时签过的那颗芯片，不能批量解锁。

右边是 **RMA Flow——Return Material Authorization**——返修流程：

1. 客户向 TI 申请 RMA。
2. TI 给客户 RMA 工具。
3. 工具用 TI 私钥签 Erase 命令。
4. HSM ROM 验证 → **擦 eFuse 中客户密钥。**
5. 锁回 GP-like 状态 → 寄回 TI。

关键意义：**客户密钥被永久销毁，TI 拿到的只是裸芯片，无任何客户信息**——这保护了客户的 IP，也让 TI 可以安心做返修分析。

Tooling & Automation（原 slide 37，约 2.5 分钟）

讲完所有概念，看工程落地——**怎么把这些机制集成到 CI/CD 里自动跑?**

第一块——OpenSSL 工具链——三步生成可签证书：

- **Step 1**：生成 BMPRK 私钥——客户根私钥，**离线保存。**
- **Step 2**：提取 BMPK 公钥 + 计算 Hash → 烧 eFuse。
- **Step 3**：签镜像 → 生成 X.509 + OID 元数据。

x509 模板里写好所有 OID——其中 `swrv` **字段每次出版本 +1。**

第二块——CI/CD 自动签名流水线——这是工程师最关心的：
4 个阶段：**build → sign → test → archive。**

关键安全约束：

- **BMPRK 私钥仅在签名服务器 HSM 中，不出网**
- 签名服务器**单独网络隔离 + 物理审计**
- CI 通过 **PKCS#11 调签名服务——而不是传 .pem 文件**
- 每次构建**自动 swrv = git tag 数字**——单调递增

ℹ️ 这个 PKCS#11 接口非常重要——**意思是 CI 把要签的数据发给签名服务器，签名服务器签完返回，私钥永远不出 HSM**——这是工业界标准做法。

第三块——反回滚 SWRV 烧 eFuse：

```
if (cert.swrv >= efuse.read_max_swrv()) {
    image_load();
    efuse.update_max_swrv(cert.swrv);
} else {
    enter_reset_loop();    // 拒绝旧版本
}
```

陷阱与对策：

- swrv 5-bit 上限 = 31 → **量产前规划好版本节奏**
- 量产首版常用 swrv=2 或 3 → 留低位空间给紧急回滚

- **一次烧错（烧到 31）整批芯片永久无法降级**

⏸️ 我再次强调——**swrv 烧 eFuse 是不可逆的，烧错就废。**这是项目里最容易出大事故的地方，**所有 PR 合到 release 分支前必须有 swrv 评审。**

Q&A（原 slide 43，2 分钟引入 + 留时间）

好，到这里所有内容就讲完了。我们快速回顾一下今天讲了什么：

第一部分——为什么要做信息安全：CIA + AAA 六大原则、汽车的六大攻击面、四个真实案例、四类攻击入口。

第二部分——业界用什么武器：加密、完整性验证、PKI/X.509、安全启动、生命周期、安全测试、国际法规。

第三部分——我们怎么落地：从威胁分析（STRIDE→CIA→对策），到芯片架构（AWR2x44），到关键澄清（HSM 两层），到密钥管理（KMS/PKS + 撤销），到 X.509 OID 详解，到 Flash 布局，到镜像创建/验证/Boot Sequence，到 Secure Debug/RMA，最后到 CI/CD 自动化。

⏸️ **如果今天大家只能记住三件事，我希望是这三件**：

1. **安全是从威胁出发倒推到对策**——不要看到酷炫的技术就盲目用，先问"这是为了防什么威胁、保护什么属性"。
2. **HSM 有两层**——TI HSM ROM 是底层硬件、Bosch HSM-RT 是应用层固件——**职责、密钥、生命周期完全不同。**
3. **eFuse 烧错就废**——swrv、Key Revision、SMPK_Hash 这些一次性烧写的字段，**项目早期一定要严格管控。**

下面是 Q&A 时间，大家有任何问题——技术上的、流程上的、踩坑经验上的——都可以提出来，我们一起讨论。

结束页（slide 44，约 30 秒）

最后还是想跟大家说一句——**信息安全不是某个团队的事，是所有人的事。**

我们写代码的时候多想一步"这里会不会有溢出"、设计架构时多问一句"这里能不能加签名"、烧 eFuse 之前多看一眼版本号——**这些日常的小动作，就是真正的信息安全文化。**

谢谢大家，今天就到这里。

时间分配总览

段落	用时	累计
开场 + 目录	2 分	2 分
第一部分（slide 4-9）	13 分	15 分
第二部分（slide 11-17）	17 分	32 分
第三部分（slide 19-37, 含调整）	26 分	58 分
Q&A 引入 + 结束	2 分	60 分

Q&A 时间额外预留 10-15 分钟（不计入 60 分钟正文）

现场小贴士

1. **slide 5（CIA+AAA）和 slide 19（STRIDE）讲完后停顿一下**，让听众消化，因为这两页的内容贯穿整个第三部分。
2. **讲到 swrv 5-bit 上限**那里**敲黑板**——这是真实事故高发区，听众应该印象最深。
3. **讲到 HSM 两层概念**那里**画两层框**——可以在白板补一个简化图，强化"上层调下层"的关系。
4. **遇到细节性的图（如 Flash Layout、Block define、Secure Boot Process 详图）——不要逐字读**，挑 2-3 个关键点讲，告诉大家"这页是参考资料，会后查"。否则容易讲超时。
5. **如果时间紧张**，可以压缩的页面：slide 6（智能化特征）、slide 16（安全测试）、slide 39/40（Flash 细节）——这些可以从 2 分钟压到 1 分钟。
6. **如果时间充裕**，可以扩展的页面：slide 38（HSM 两层）、slide 31/32（OID 字段详解）——这些是听众最容易混淆、最需要反复讲的。